

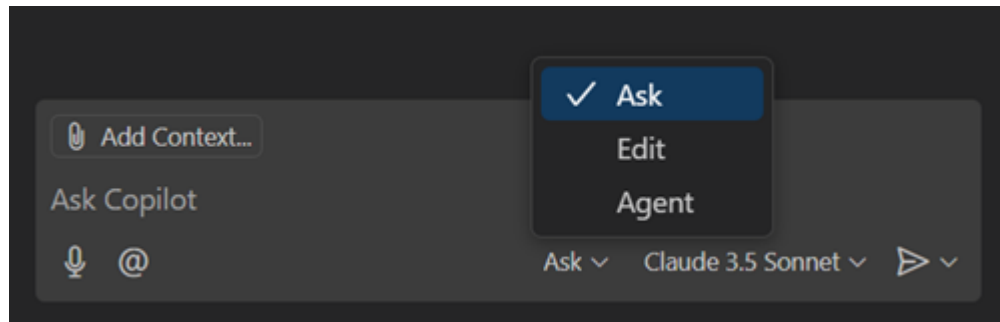
Exercise: AI Assisted Coding

Copilot Chat modes

Ask	Conversational Assistance The standard chat interface where you ask questions, request explanations, or get code snippets without modifying files directly.	• Understanding existing code • Debugging concepts • Generating small snippets to copy-paste
Plan	Architecture & Strategy Analyzes your request and codebase to build a step-by-step implementation plan (blueprint) without writing code immediately.	• Breaking down complex features • clarifying requirements before coding • Designing large refactors
Edit	Controlled Modifications Allows you to select specific files and instruct Copilot to apply granular changes directly to them.	• Refactoring a specific file • Fixing a known bug in a function • Cleaning up code logic
Agent	Autonomous Execution An autonomous mode that can plan, edit, run terminal commands, and fix errors across multiple files to complete a task from start to finish.	• End-to-end feature implementation • Complex multi-file tasks • Tasks requiring iterative testing and fixing

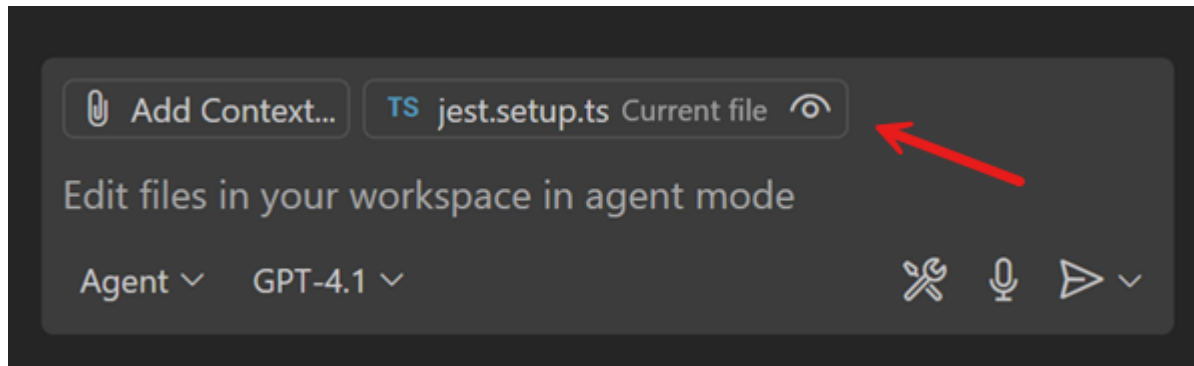
Ask Mode

- Answering your question without touching your code.
 - Explain what the code does, suggest how to test it, give you a code snippet that implements what you are asking about
 - How to use a certain library, how to structure a SQL query, etc



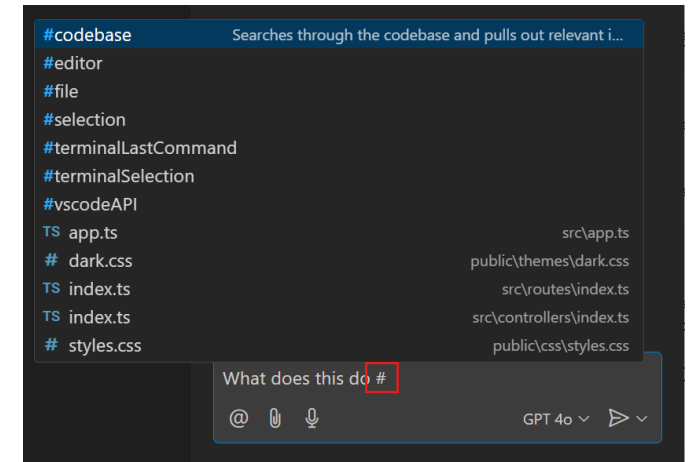
Implicit Context

- VS Code automatically adds the active file as a context item in the chat input box (indicated by Current file)

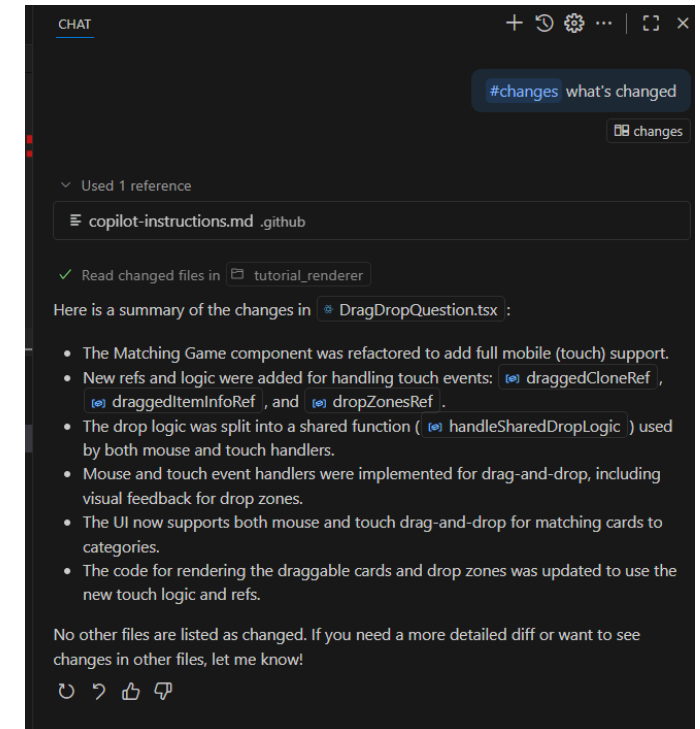


#-mentions

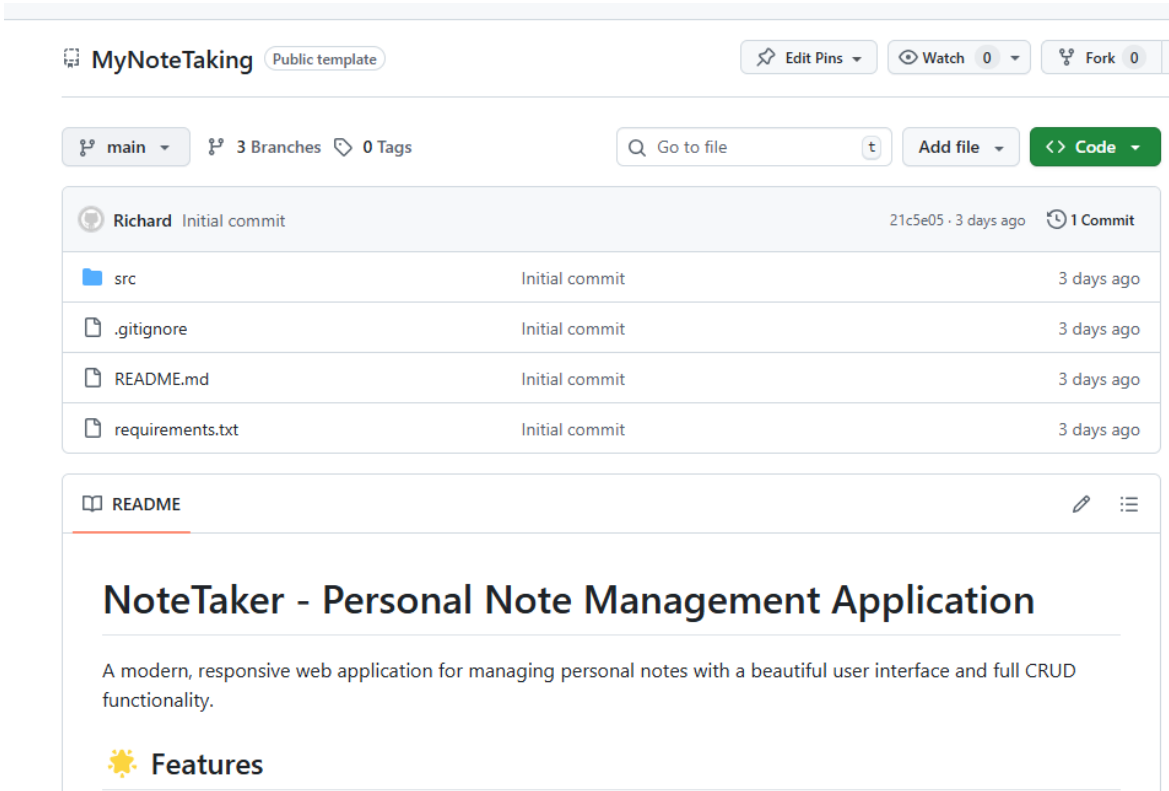
- # followed by the file name, folder name, or symbol name
- #codebase
 - perform a codebase search for your workspace.
- #fetch
 - Reference contents from the web (e.g. updated API doc)



Category	Examples
Reference your pending source control changes	- "Summarize the #changes" - "Generate release notes based on the #changes"
Understand the codebase	- "Explain how authentication works in #codebase" - "Where is the database connecting string configured? #codebase" - "How do I build this #codebase?" - "Where is #getUser used? #usages"
Generate code that is consistent with your codebase	- "Create an about page and include it in the nav bar #codebase" - "Add a new API route for updating the address info #codebase" - "Add a login button and style it based on #styles.css"
Fix issues in the workspace	- "Fix the issues in #problems" - "Fix the failing tests #testFailure"
Reference content from the web	- "How do I use the useState hook in react 18? #fetch https://18.react.dev/reference/react/useState#usage" - "Build an API endpoint to fetch address info, use the template from #githubRepo contoso/api-templates"



The note taking app (Generated with Manus AI)



The screenshot shows the GitHub repository page for 'MyNoteTaking'. At the top, there are buttons for 'Edit Pins', 'Watch' (0), and 'Fork' (0). Below these, there's a navigation bar with 'main' selected, '3 Branches', and '0 Tags'. A search bar 'Go to file' and buttons 'Add file' and '<> Code' are also present. The main content area shows a commit by 'Richard' (Initial commit) from 3 days ago. Below the commit, there's a list of files: 'src', '.gitignore', 'README.md', and 'requirements.txt', all with 'Initial commit' and '3 days ago' timestamps. At the bottom, there's a 'README' section with the title 'NoteTaker - Personal Note Management Application' and a description: 'A modern, responsive web application for managing personal notes with a beautiful user interface and full CRUD functionality.' Below the description, there's a 'Features' section with a sun icon.

MyNoteTaking Public template

Edit Pins Watch 0 Fork 0

main 3 Branches 0 Tags

Go to file Add file <> Code

Richard Initial commit 21c5e05 · 3 days ago 1 Commit

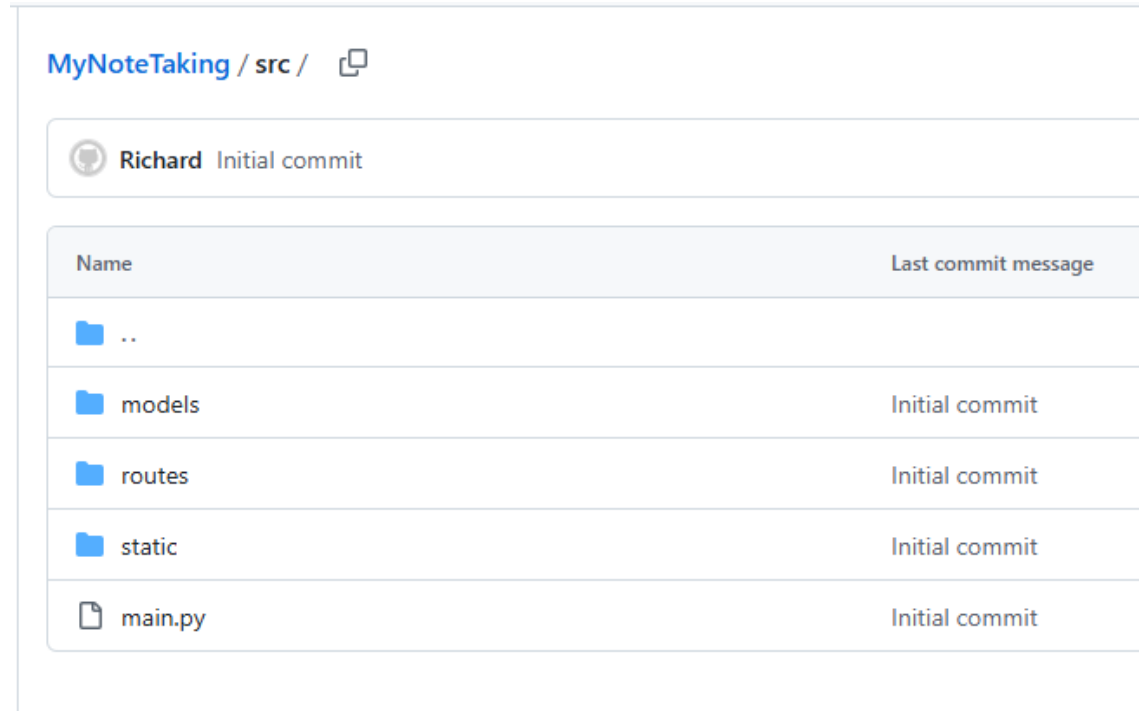
src	Initial commit	3 days ago
.gitignore	Initial commit	3 days ago
README.md	Initial commit	3 days ago
requirements.txt	Initial commit	3 days ago

README

NoteTaker - Personal Note Management Application

A modern, responsive web application for managing personal notes with a beautiful user interface and full CRUD functionality.

Features



The screenshot shows the file structure of the 'MyNoteTaking' repository. The path 'MyNoteTaking / src /' is displayed at the top. Below it, there's a commit by 'Richard' (Initial commit). A table lists the files and folders in the repository, showing their names and the last commit message.

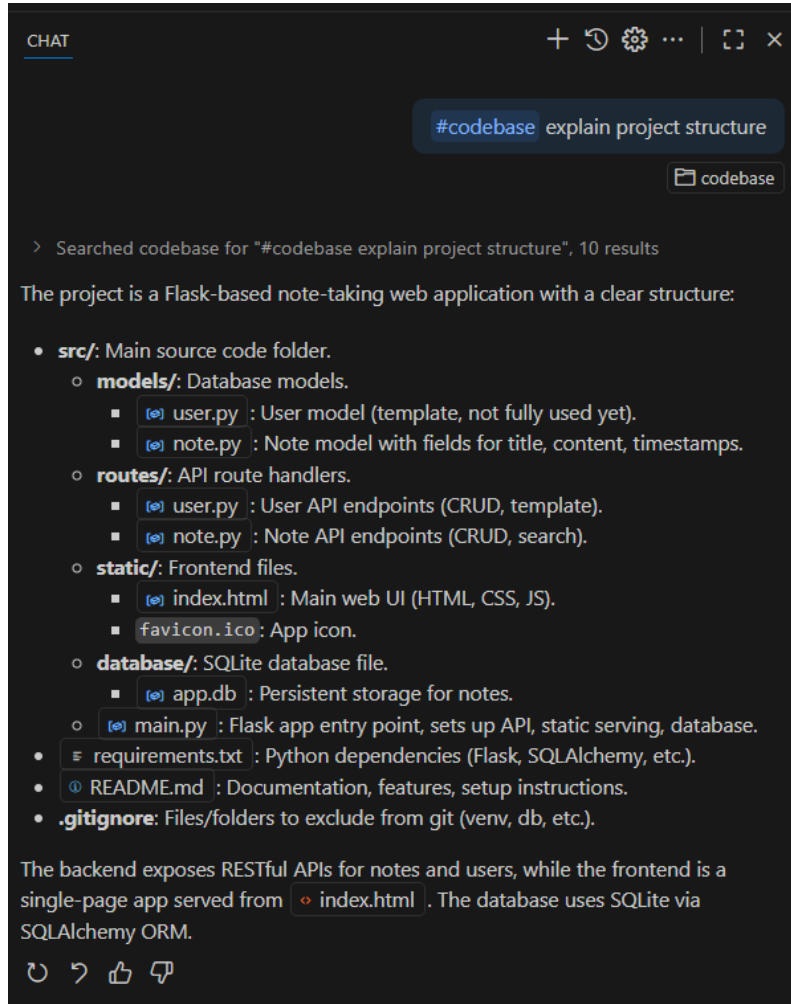
MyNoteTaking / src /

Richard Initial commit

Name	Last commit message
..	
models	Initial commit
routes	Initial commit
static	Initial commit
main.py	Initial commit

The note taking app developed by Manus AI in lecture 1:
<https://3dhkiloc88dkk.manus.space/>

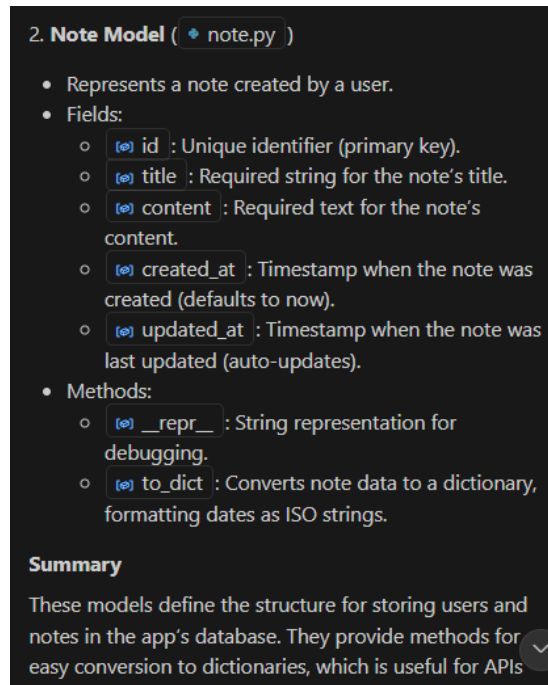
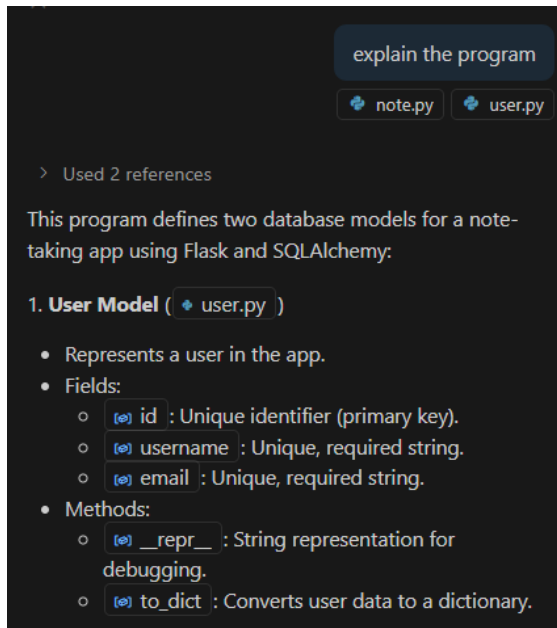
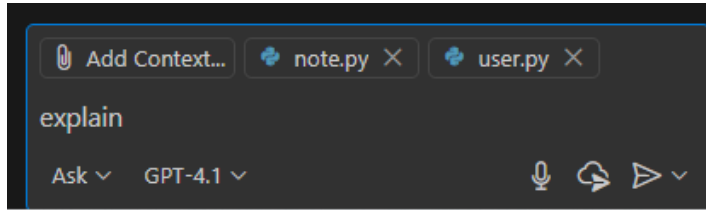
Let's explore the codebase



#codebase

A chat variable used in Copilot Chat that references relevant context from your entire workspace or codebase and adds that context to your chat prompt.

- Add context to ask questions

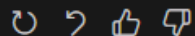


which part of the code handle creating notes

The part of the code that handles creating notes is typically found in the route handler for POST requests to the notes endpoint. In your codebase, this is likely in:

```
@note_bp.route('/notes', methods=['POST'])
def create_note():
    # ...existing code...
    # Logic for creating a new note (e.g.
    # ...existing code...
```

This function receives the note data from the client, processes it, and adds the new note to your storage (such as a database).



VS

which part of the code handle creating notes

#codebase

codebase

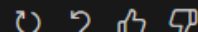
> Searched codebase for "which part of the code handle creating notes #codebase ", 10 results

The part of the code that handles creating notes is in `note.py`, specifically the `create_note` function:

```
@note_bp.route('/notes', methods=['POST'])
def create_note():
    """Create a new note"""
    try:
        data = request.json
        if not data or 'title' not in data:
            return jsonify({'error': 'Title is required'})

        note = Note(title=data['title'],
                    db.session.add(note)
                    db.session.commit()
                    return jsonify(note.to_dict()), 201
    except Exception as e:
        db.session.rollback()
        return jsonify({'error': str(e)})
```

This function receives a POST request to `/api/notes`, validates the input, creates a new `Note` object, saves it to the database, and returns the created note as JSON.



Starting the app

```
PS C:\programs\notetaking-app\src> python main.py
* Serving Flask app 'main'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use
a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5001
* Running on http://158.132.10.133:5001
Press CTRL+C to quit
* Restarting with watchdog (windowsapi)
* Debugger is active!
* Debugger PIN: 296-050-124
```

- Test the app using the exposed URL by codespaces

PORTS 1				
Port	Forwarded Address	Running Process	Visibility	Origin
5001	https://reimagined-co...	/home/codespace/.python/current/bin/python s...	Private	Auto Forwarded
Add Port				

- Test the app and observe the backend logs

```
a production WSGI server instead.
```

```
* Running on all addresses (0.0.0.0)
```

```
* Running on http://127.0.0.1:5001
```

```
* Running on http://158.132.10.133:5001
```

```
Press CTRL+C to quit
```

```
* Restarting with watchdog (windowsapi)
```

```
* Debugger is active!
```

```
* Debugger PIN: 296-050-124
```

```
127.0.0.1 - - [04/Sep/2025 12:35:00] "GET / HTTP/1.1" 200 -
```

```
127.0.0.1 - - [04/Sep/2025 12:35:01] "GET /api/notes HTTP/1.1" 200 -
```

```
127.0.0.1 - - [04/Sep/2025 12:35:01] "GET /favicon.ico HTTP/1.1" 200 -
```

```
127.0.0.1 - - [04/Sep/2025 12:35:09] "POST /api/notes HTTP/1.1" 201 -
```

```
127.0.0.1 - - [04/Sep/2025 12:35:09] "PUT /api/notes/3 HTTP/1.1" 200 -
```

```
127.0.0.1 - - [04/Sep/2025 12:35:12] "GET / HTTP/1.1" 304 -
```

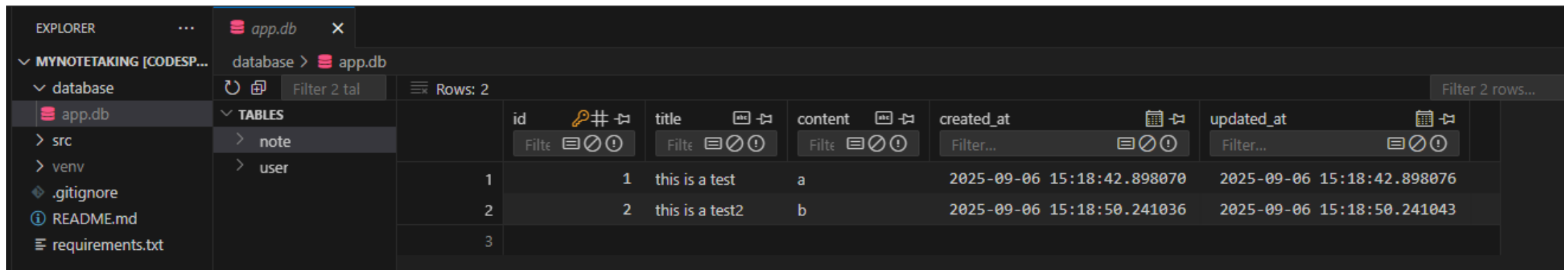
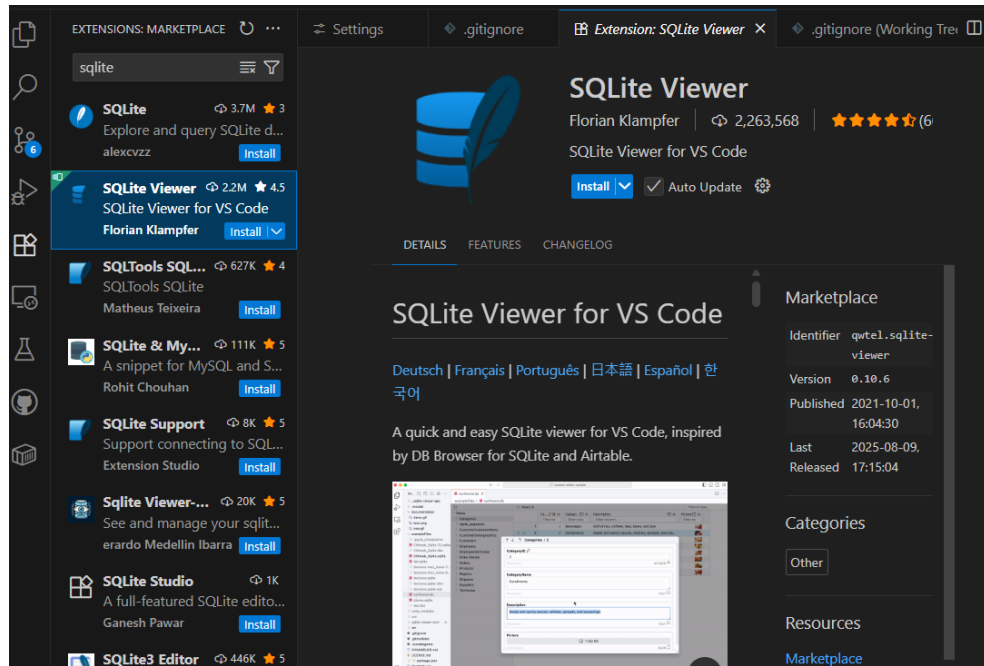
```
127.0.0.1 - - [04/Sep/2025 12:35:12] "GET /api/notes HTTP/1.1" 200 -
```

```
127.0.0.1 - - [04/Sep/2025 12:35:13] "GET /favicon.ico HTTP/1.1" 304 -
```

```
127.0.0.1 - - [04/Sep/2025 12:35:19] "POST /api/notes HTTP/1.1" 201 -
```

```
127.0.0.1 - - [04/Sep/2025 12:35:20] "PUT /api/notes/4 HTTP/1.1" 200 -
```

- Install “SQLite Viewer” VSCode plugin
- View the database



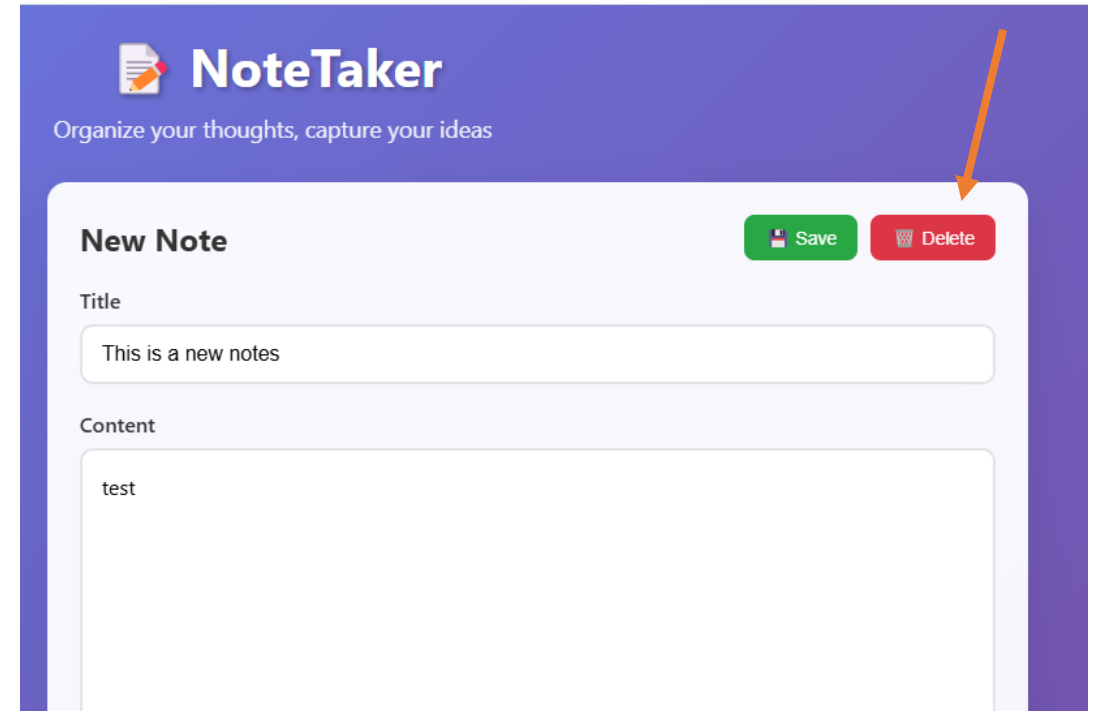
Edit Mode

- You can pick the file(s) you want to change and describe the update in natural language. Copilot will apply inline, review-ready code edits across those files.
- Edit mode is perfect when you know what you want to do but don't necessarily want to write it all out yourself.
- **Exercise:** Use the edit mode to change the background color of the web app
 - select `static/index.html` as context

Exercices: Fix a bug

- Use the Copilot **Edit** mode to fix the bug that delete button should not appear before the notes is saved.

```
src > static > index.html > ...
2 <html lang="en">
326 <body>
374 <script>
375 class NoteTaker {
473 // Remove active state from all notes
474 document.querySelectorAll('.note-item').forEach(item => {
475   item.classList.remove('active');
476 });
477 }
478
479 showEditor() {
480   document.getElementById('emptyState').style.display = 'none';
481   document.getElementById('editorForm').style.display = 'block';
482   document.getElementById('editorActions').style.display = 'flex';
483
484   // Hide delete button for new notes (no id)
485   const deleteBtn = document.getElementById('deleteBtn');
486   if (this.currentNote && this.currentNote.id) {
487     deleteBtn.style.display = '';
488   } else {
489     deleteBtn.style.display = 'none';
490   }
491 }
492
493 hideEditor() {
494   document.getElementById('emptyState').style.display = 'block';
495   document.getElementById('editorForm').style.display = 'none';
496 }
```



Git

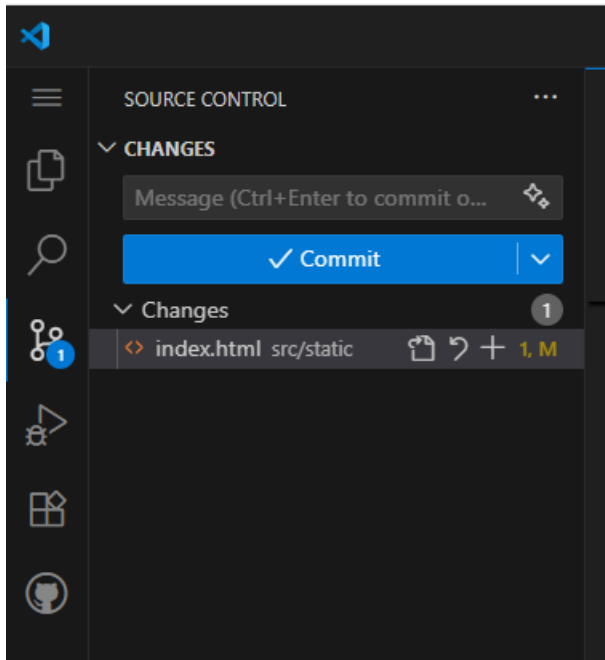
- In GitHub Codespaces, any changes made within the environment will be lost once the environment is deleted because it is temporary.
- To ensure your work is saved, you should first commit your changes to the local Git repository. After committing, push these changes to the remote repository on GitHub.
- This process ensures that all your work is stored safely and can be accessed from any future environment.

- Git: Version control system used to track changes in source code during software development
- Commit: A commit in Git is a snapshot of your changes in the repository.

Git Branch

- **Branch:** A parallel version of the repository. It allows you to work on different features, fixes, or experiments independently from the main codebase.
- In Git, the use of branches allows you to diverge from the main line of development and continue to do work without affecting the main branch.
 - E.g., you may work on new features, bug fixes, or experiments without affecting the main codebase. Multiple developers can work on different tasks simultaneously without interfering with each other

- Create a new branch “Dev” and checkout to the branch
- Stage and commit the change with a descriptive commit message
- Push the new commit to the GitHub repository

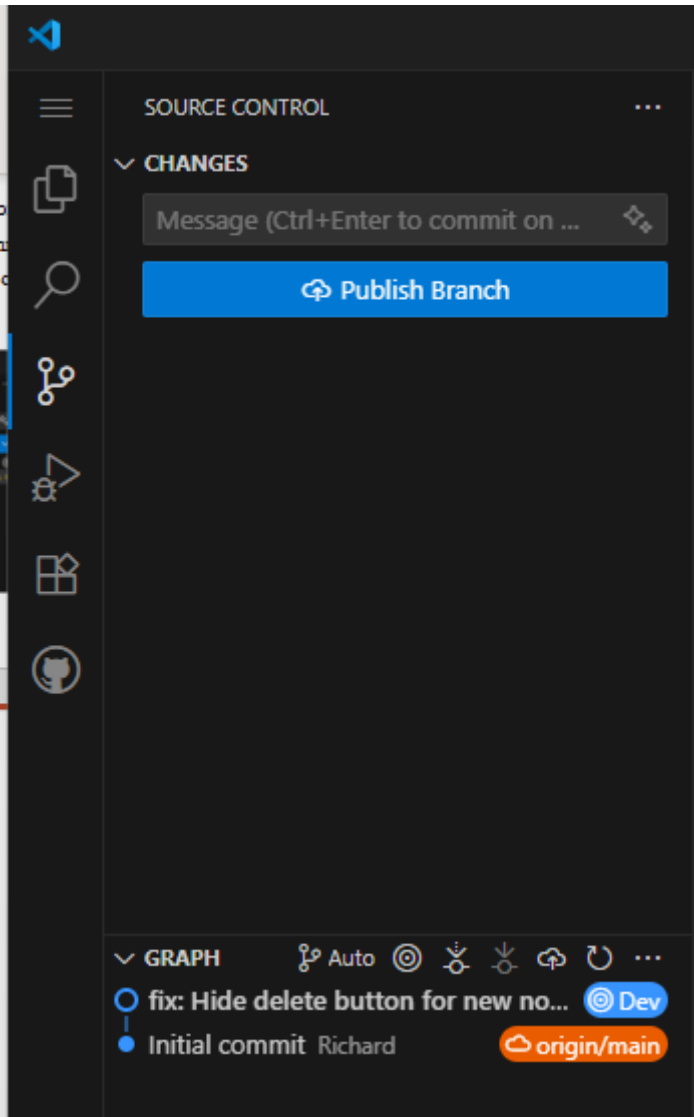


```
# Create and checkout to a new branch named "Dev"
git checkout -b Dev

# Stage your changes
git add .

# Commit with a descriptive message
git commit -m "fix: Hide delete button for new notes in the editor"

# Push the new branch and commit to GitHub
git push origin Dev
```



```
# Push the new branch and commit to GitHub  
git push origin Dev
```

- Now, generate a pull request
 - A pull request is a formal proposal to merge code changes from one branch into another, allowing for review, discussion, and approval before integration.
- Request Copilot to generate a summary

 Dev had recent pushes 53 seconds ago

Compare & pull request



Add a title

Fix: Newline rendering in notes list

Add a description

WritePreview



Add your description here...

Generate

Summary

Generate a summary of the changes in this pull request.

 Markdown is supported Paste, drop, or click to add files

Create pull request

- You may request a team member/copilot to review the code change

 **Add a title**

Fix: Newline rendering in notes list

Add a description

Write Preview

This pull request makes a small but important improvement to the note preview rendering in the `src/static/index.html` file. The change ensures that line breaks in note content are displayed correctly in the note preview section.

* Note preview content now replaces newline characters with `
` tags, allowing multi-line notes to be displayed with proper formatting in the preview. [[1]](diffhunk://#diff-6c71709b4d31c8f650dbe64ffc24d55cede1d64448e827c4b1292ffeb2a3acc3L439-R441) [[2]](diffhunk://#diff-6c71709b4d31c8f650dbe64ffc24d55cede1d64448e827c4b1292ffeb2a3acc3L592-R596)

 Markdown is supported  Paste, drop, or click to add files

Reviewers 

Suggestions

 Copilot [Request](#)

Assignees 

No one—[assign yourself](#)

Labels 

None yet

Projects 

None yet

Milestone 

No milestone

Development

Use [Closing keywords](#) in the description to automatically close issues

Helpful resources

[GitHub Community Guidelines](#)

[Create pull request](#)

 Remember, contributions to this repository should follow our [GitHub Community Guidelines](#).

- Review the suggestion of the Copilot coding agent.
- Merge the pull request

Copilot AI reviewed 2 minutes ago

View reviewed changes

Copilot AI left a comment

Pull Request Overview

This PR fixes newline rendering in note preview displays by converting newline characters to HTML `<br>` tags. This ensures that multi-line notes display with proper line breaks in the preview section instead of appearing as single-line text.

- Modifies note preview rendering to replace `\n` characters with `<br>` tags
- Applied consistently to both note list rendering sections in the HTML file

Tip: Customize your code reviews with copilot-instructions.md. [Create the file](#) or [learn how to get started](#).

```
src/static/index.html
... @@ -436,7 +436,9 @@ <h3>Welcome to NoteTaker!</h3>
436 436         <div class="note-item" ${this.currentNote.id === note.id ? 'active' : ''}
437 437         data-note-id="${note.id}" onclick="noteTaker.selectNote(${note.id})">
438 438         <div class="note-title">${this.escapeHtml(note.title || 'Untitled')}</div>
439 -         <div class="note-preview">${this.escapeHtml(note.content || 'No content')}</div>
439 +         <div class="note-preview">${
440 +         this.escapeHtml(note.content || 'No content').replace(/\n/g, '<br>')}

```

Copilot AI 2 minutes ago

The newline replacement with `<br>` tags happens after HTML escaping, which means the `<br>` tags will be displayed as literal text instead of functioning as line breaks. Move the newline replacement before `escapeHtml()` or use a different approach to preserve line breaks while maintaining security.

Suggested change

```
440 -         this.escapeHtml(note.content || 'No content').replace(/\n/g, '<br>')}
440 +         this.escapeHtml(note.content || 'No content').replace(/\n/g, '<br>'))

```

Commit suggestion Add suggestion to batch

No conflicts with base branch
Merging can be performed automatically.

Merge pull request You can also merge this with the command line. [View command line instructions](#).

Create a merge commit
All commits from this branch will be added to the base branch via a merge commit.

Squash and merge
The 3 commits from this branch will be combined into one commit in the base branch.

Rebase and merge
The 3 commits from this branch will be rebased and added to the base branch.

Commit message

Merge pull request #1 from PolyUTest/Dev

Extended description

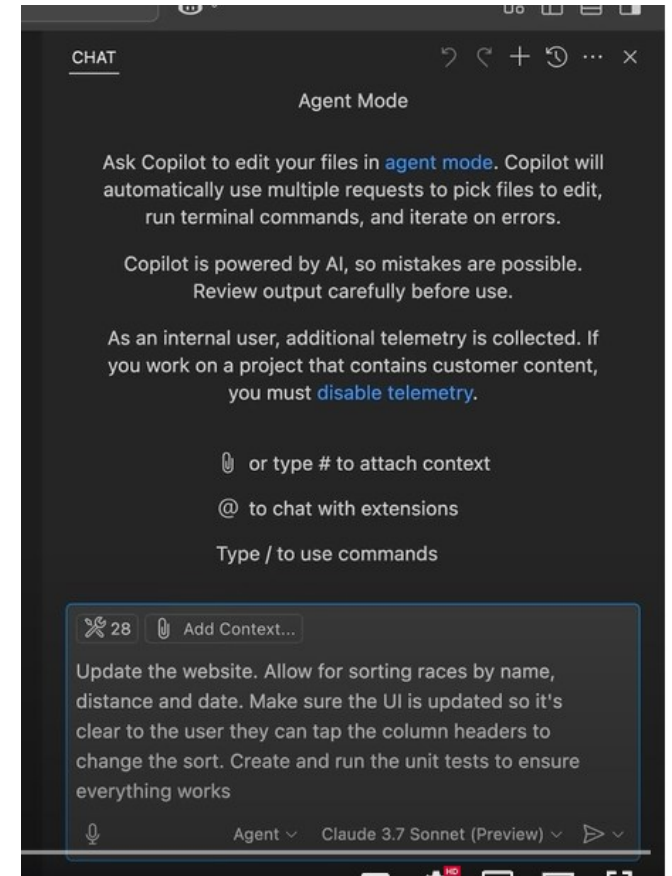
Fix Newline rendering in notes list

This commit will be authored by 218137+cswdui@users.noreply.github.com.

Confirm merge Cancel

Agent mode

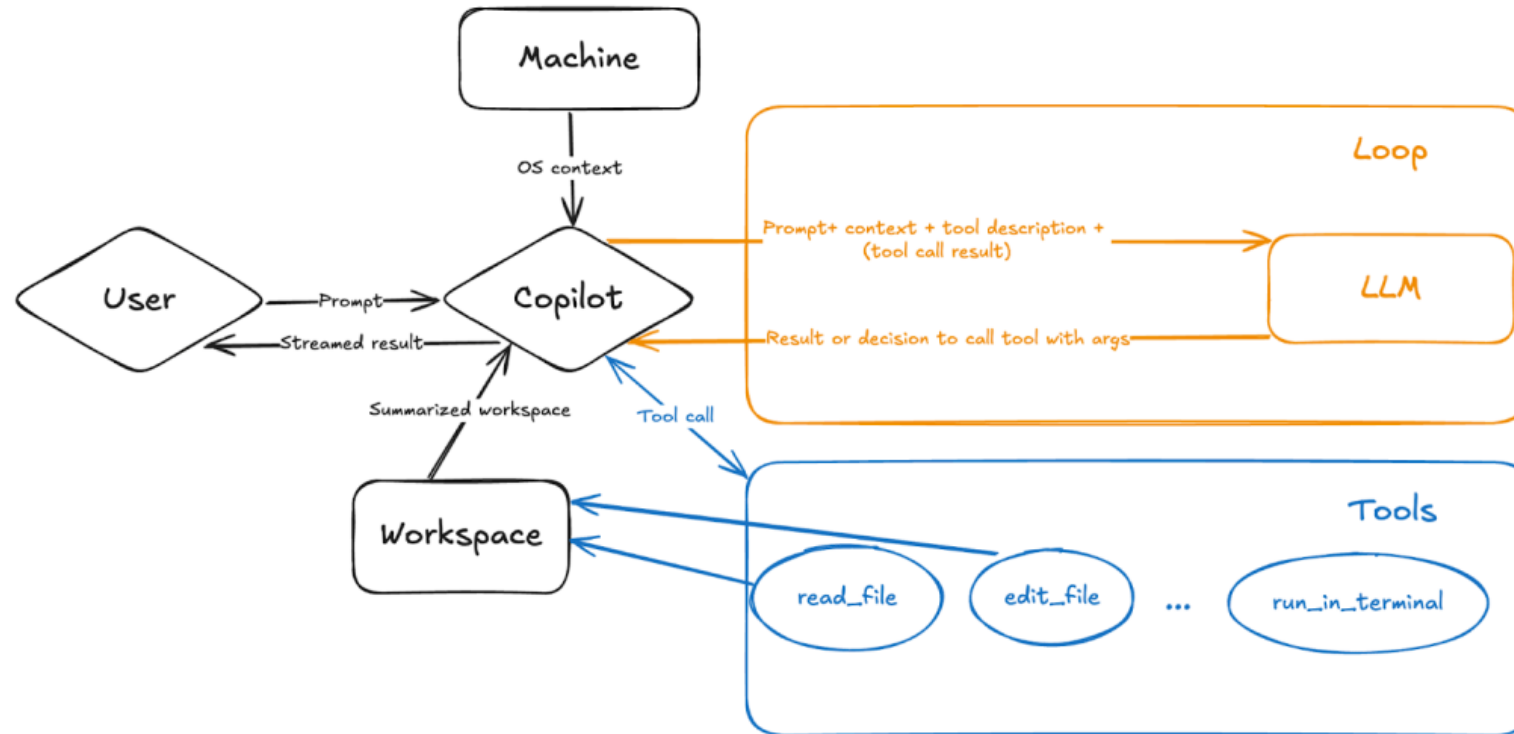
- Use natural language to specify a high-level task
- Let AI it autonomously plans and executes a multi-step workflow to complete those tasks across your codebase
 - Analyze your codebase to grasp the full context.
 - Plan and execute multi-step solutions.
 - Run commands or tests.
 - Reach out to external tools for specialized tasks (e.g. MCP)
 - Monitoring the results of code changes, build outcomes, test failures, and terminal outputs.
 - Iterating and refining the changes autonomously until the task is successfully completed or until it requires input from you.



<https://github.blog/ai-and-ml/github-copilot/agent-mode-101-all-about-github-copilots-powerful-mode/>

[Video](#)

How does agent mode in GitHub Copilot work?

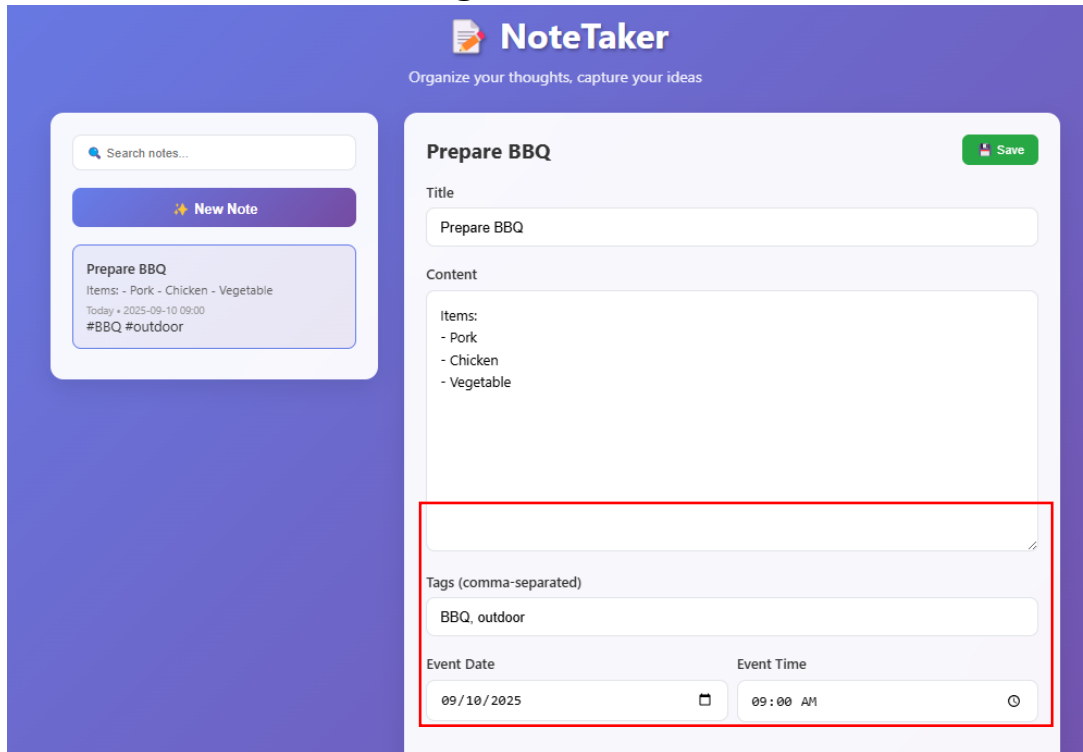


- User query is augmented with workspace structure summary, tool descriptions, other contexts.
- Copilot parses the question & plans resolution
- AI begins working and uses tools autonomously to reach the goal
 - Searching workspace, Reading/Editing files, Running terminal commands, Handling errors and applying fixes
- When errors or issues (like syntax errors or failed tests) are encountered, AI will analyze the problem and make changes to fix those errors and move closer to the desired result.
 - E.g. editing code, running new commands, or using different tools

Add new fields for the notes item

- First, use the “Plan” mode to generate the plan

Task: add the following fields in the notes



The screenshot shows the NoteTaker application interface. On the left, there is a sidebar with a search bar, a 'New Note' button, and a list of notes. The main area displays a note titled 'Prepare BBQ'. The note has a title field, a content field, a tags field, and an event date/time field. The tags field is highlighted with a red box. The event date/time field is also highlighted with a red box.

NoteTaker
Organize your thoughts, capture your ideas

Search notes...

New Note

Prepare BBQ
Items: - Pork - Chicken - Vegetable
Today • 2025-09-10 09:00
#BBQ #outdoor

Prepare BBQ

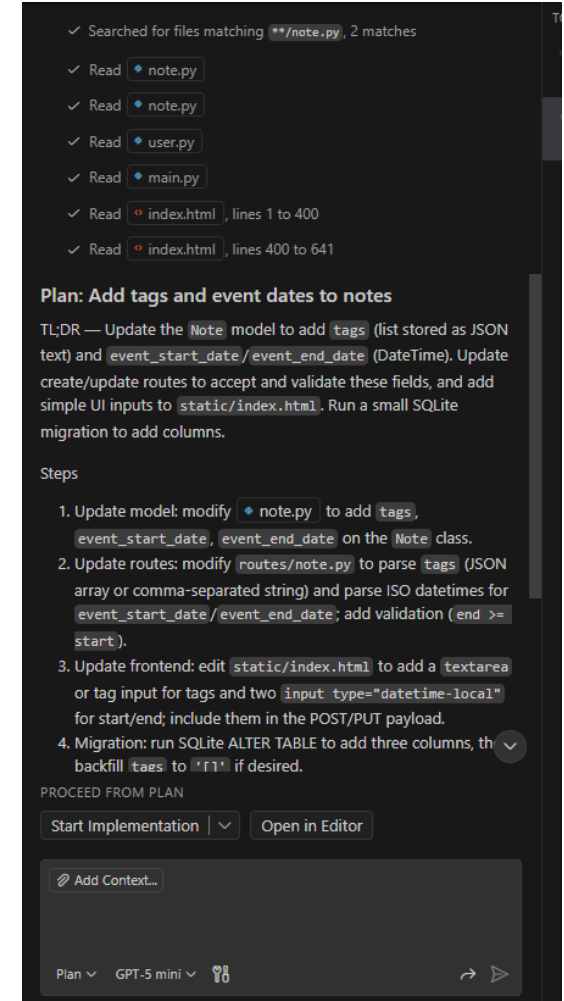
Title
Prepare BBQ

Content
Items:
- Pork
- Chicken
- Vegetable

Tags (comma-separated)
BBQ, outdoor

Event Date
09/10/2025

Event Time
09:00 AM



The screenshot shows the Plan mode interface. It displays a list of files searched for matching patterns, a plan to add tags and event dates to notes, and a list of steps to implement the plan.

✓ Searched for files matching `**/note.py`, 2 matches

✓ Read `note.py`

✓ Read `note.py`

✓ Read `user.py`

✓ Read `main.py`

✓ Read `index.html`, lines 1 to 400

✓ Read `index.html`, lines 400 to 641

Plan: Add tags and event dates to notes

TL;DR — Update the `Note` model to add `tags` (list stored as JSON text) and `event_start_date`/`event_end_date` (DateTime). Update create/update routes to accept and validate these fields, and add simple UI inputs to `static/index.html`. Run a small SQLite migration to add columns.

Steps

1. Update model: modify `note.py` to add `tags`, `event_start_date`, `event_end_date` on the `Note` class.
2. Update routes: modify `routes/note.py` to parse `tags` (JSON array or comma-separated string) and parse ISO datetimes for `event_start_date`/`event_end_date`; add validation (`end >= start`).
3. Update frontend: edit `static/index.html` to add a `textarea` or tag input for tags and two `input type="datetime-local"` for start/end; include them in the POST/PUT payload.
4. Migration: run SQLite ALTER TABLE to add three columns, then backfill `tags` to `['']` if desired.

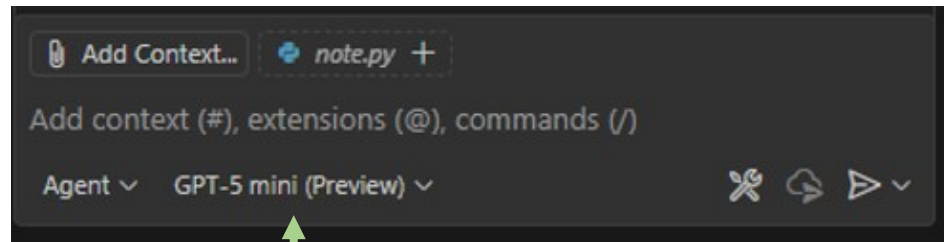
PROCEED FROM PLAN

Start Implementation | Open in Editor

Add Context...

Plan ▾ GPT-5 mini ▾

- Use GitHub Copilot “**Agent**” mode to implement it
- Observe how the agentic coding works.



Task: add the following fields in the notes

NoteTaker
Organize your thoughts, capture your ideas

Search notes...

New Note

Prepare BBQ Save

Title
Prepare BBQ

Content
Items: - Pork - Chicken - Vegetable
Today • 2025-09-10 09:00
#BBQ #outdoor

Tags (comma-separated)
BBQ, outdoor

Event Date
09/10/2025

Event Time
09:00 AM

database > app.db

Filter 2 tables...

TABLES

- note
- user

	id	title	content	created_at	updated_at	tags	event_date	event_time
1	1	Prepare BBQ	Items: - Pork - Chicken - Vegetable	2025-09-06 16:01:48.508587	2025-09-06 16:01:48.508592	["BBQ", "outdoor"]	2025-09-10	09:00

Agent mode vs edit mode

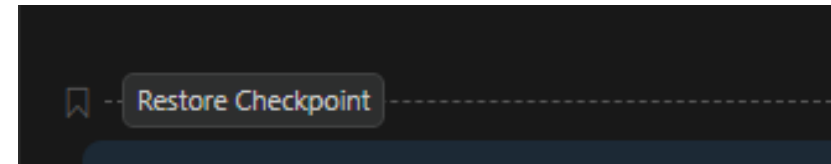
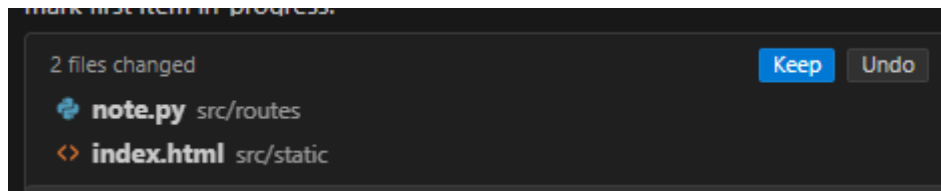
- **Edit scope:** agent mode autonomously determines the relevant context and files to edit. In edit mode, you need to specify the context yourself.
- **Task complexity:** agent mode is better suited for complex tasks that require not only code edits but also the invocation of tools and terminal commands.
- **Duration:** agent mode involves multiple steps to process a request, so it might take longer to get a response.
 - E.g., to determine the relevant context and files to edit, determine the plan of action.
- **Self-healing:** agent mode evaluates the outcome of the generated edits and might iterate multiple times to resolve intermediate issues.
- **Request quota:** in agent mode, depending on the complexity of the task, one prompt might result in many requests to the backend

Which mode to use?

- Agent mode might
 - Touch a file you thought you had agreed to leave alone.
 - Suggest running a command you don't expect.
 - It takes longer for reasoning and performing the different steps, especially on bigger projects.
- Sometimes you'll be working on part of a codebase that needs to be handled with more precision.
 - E.g. Treaking a couple of files or making a targeted change in a sensitive system
 - Ask or edit mode might be the better fit

Undoing the changes generated by AI

- “Undo” the AI changes
- “Restore Checkpoint” in the GitHub Copilot Chat
- Using “git reset” Command
 - Soft, Mixed vs. Hard reset

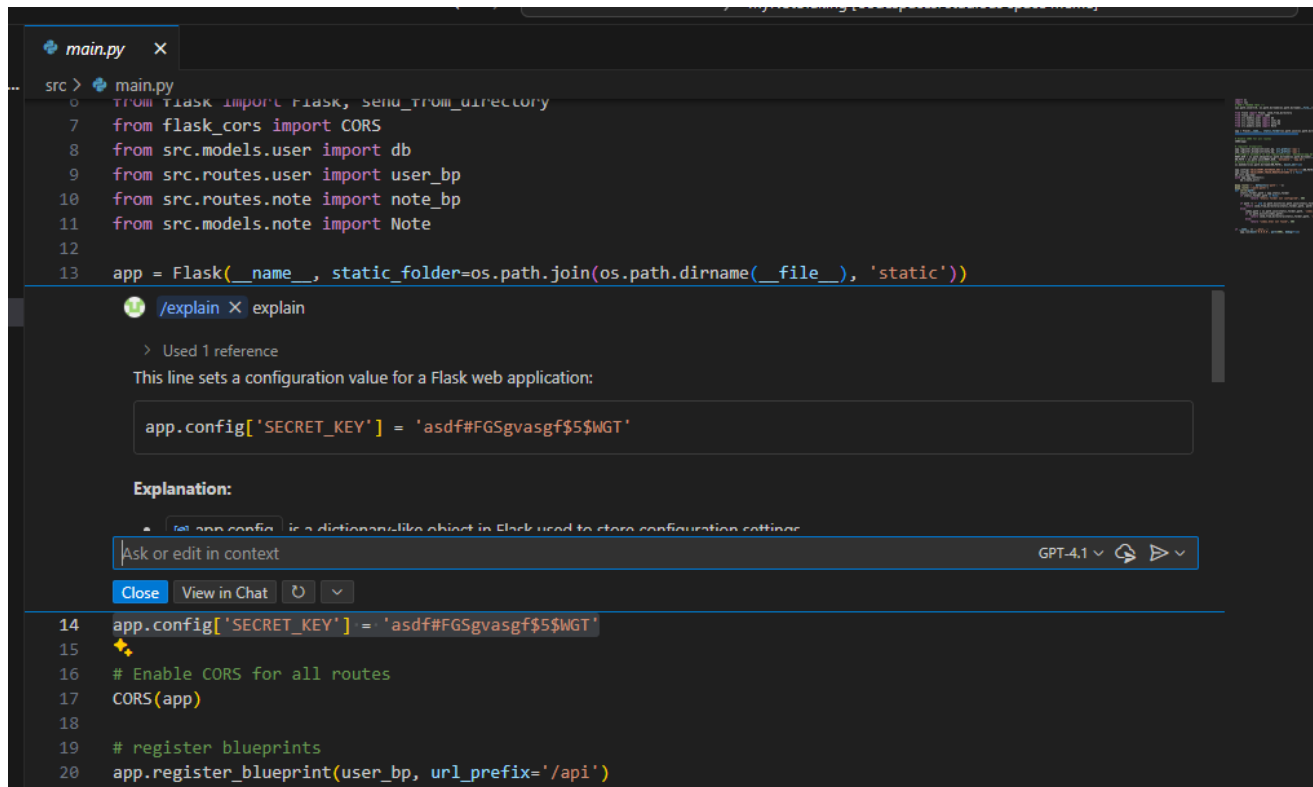


Exercise

- Fix the remaining bugs of the app.
- Add new features to the app.
- Remark:
 - After the AI makes the code changes, you should test to see if the code changes is working correctly
 - Commit each change in the “Dev” branch
 - Push the commits and update the remote repository (GitHub)

Inline chat

- Ask questions and get suggestions directly in the editor or get help with shell commands within the integrated terminal.
- Open editor inline chat by using the **Ctrl+I** keyboard shortcut



The screenshot shows a VS Code editor with a file named `main.py`. The code is a Flask web application. An inline chat window is open, triggered by the `/explain` command. The chat window shows the following content:

```
src > main.py
0 from flask import Flask, send_from_directory
7 from flask_cors import CORS
8 from src.models.user import db
9 from src.routes.user import user_bp
10 from src.routes.note import note_bp
11 from src.models.note import Note
12
13 app = Flask(__name__, static_folder=os.path.join(os.path.dirname(__file__), 'static'))
```

The chat window displays the following explanation:

Used 1 reference

This line sets a configuration value for a Flask web application:

```
app.config['SECRET_KEY'] = 'asdf#FGSgvasgf$5$WGT'
```

Explanation:

`app.config` is a dictionary-like object in Flask used to store configuration settings.

Ask or edit in context

Close View in Chat

The chat window is powered by GPT-4.1.

```
14 app.config['SECRET_KEY'] = 'asdf#FGSgvasgf$5$WGT'
15
16 # Enable CORS for all routes
17 CORS(app)
18
19 # register blueprints
20 app.register_blueprint(user_bp, url_prefix='/api')
```

Next edit suggestions (NES)

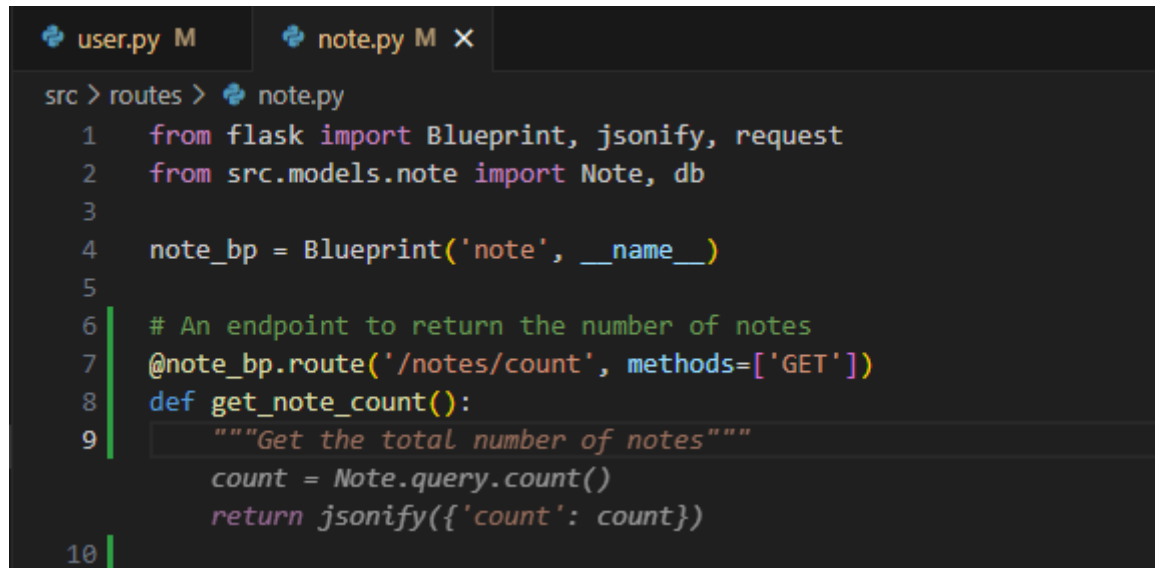
- Based on the edits you're making, NES both predicts the location of the next edit you'll want to make and what that edit should be.
- Press tab to accept the suggestion

```
src > routes > note.py
1  from flask import Blueprint, jsonify, request
2  from src.models.note import Note, db
3
4  note_blueprint = Blueprint('note', __name__)
5
6  @note_bp.route('/notes', methods=['GET'])
7  def get_blueprint
8      """Get all notes, ordered by most recently updated"""
9      notes = Note.query.order_by(Note.updated_at.desc()).all()
10     return jsonify([note.to_dict() for note in notes])
11
```

<https://code.visualstudio.com/docs/copilot/ai-powered-suggestions>

Code completions

- Start typing in the editor, and Copilot provides code suggestions that match your coding style and take your existing code into account.
- Generate suggestions from code comments
- Press **tab** to accept the suggestion



```
src > routes > note.py
1 from flask import Blueprint, jsonify, request
2 from src.models.note import Note, db
3
4 note_bp = Blueprint('note', __name__)
5
6 # An endpoint to return the number of notes
7 @note_bp.route('/notes/count', methods=['GET'])
8 def get_note_count():
9     """Get the total number of notes"""
10     count = Note.query.count()
11     return jsonify({'count': count})
```


Let's develop a new feature to translate the notes using GPT 4.1-mini

Prepare BBQ Chinese Translate Save Delete

Title

Prepare BBQ

Content

Items:

- Pork
- Chicken
- Vegetable

Tags (comma-separated)

BBQ, outdoor

Event Date

09/10/2025

Event Time

09:00 AM

准备烧烤 Chinese Translate Save Delete

Title

准备烧烤

Content

物品：

- 猪肉
- 鸡肉
- 蔬菜

Tags (comma-separated)

BBQ, outdoor

Event Date

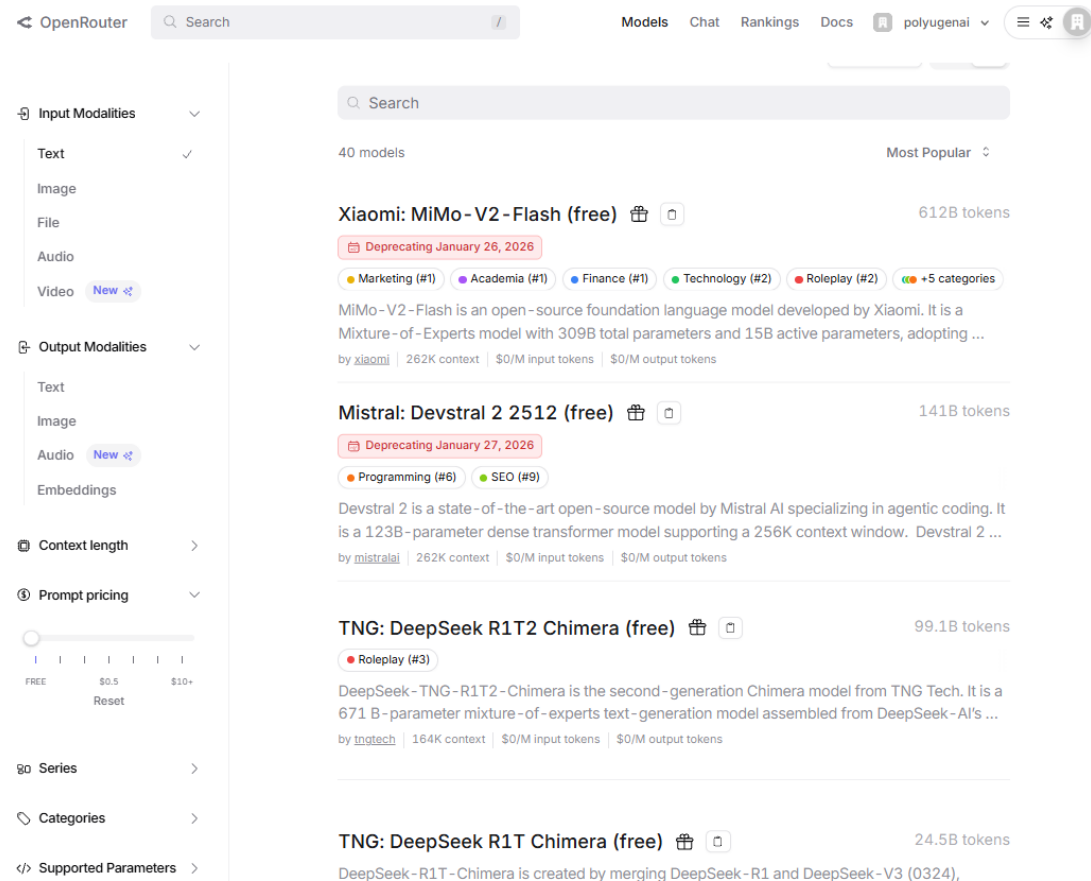
09/10/2025

Event Time

09:00 AM

Openrouter APIs

- Free models are available
- Free quote
 - Budget can get to ~\$1



The screenshot displays the OpenRouter website interface. On the left, there is a sidebar with filters for Input Modalities (Text, Image, File, Audio, Video), Output Modalities (Text, Image, Audio, Embeddings), Context length, Prompt pricing, Series, Categories, and Supported Parameters. The main content area shows a search bar and a list of models. The first model listed is 'Xiaomi: MiMo-V2-Flash (free)' with 612B tokens, marked as deprecated on January 26, 2026. The second model is 'Mistral: Devstral 2 2512 (free)' with 141B tokens, marked as deprecated on January 27, 2026. The third model is 'TNG: DeepSeek R1T2 Chimera (free)' with 99.1B tokens, marked as deprecated on January 27, 2026. The fourth model is 'TNG: DeepSeek R1T Chimera (free)' with 24.5B tokens. Each model entry includes a brief description and a link to the model's page.

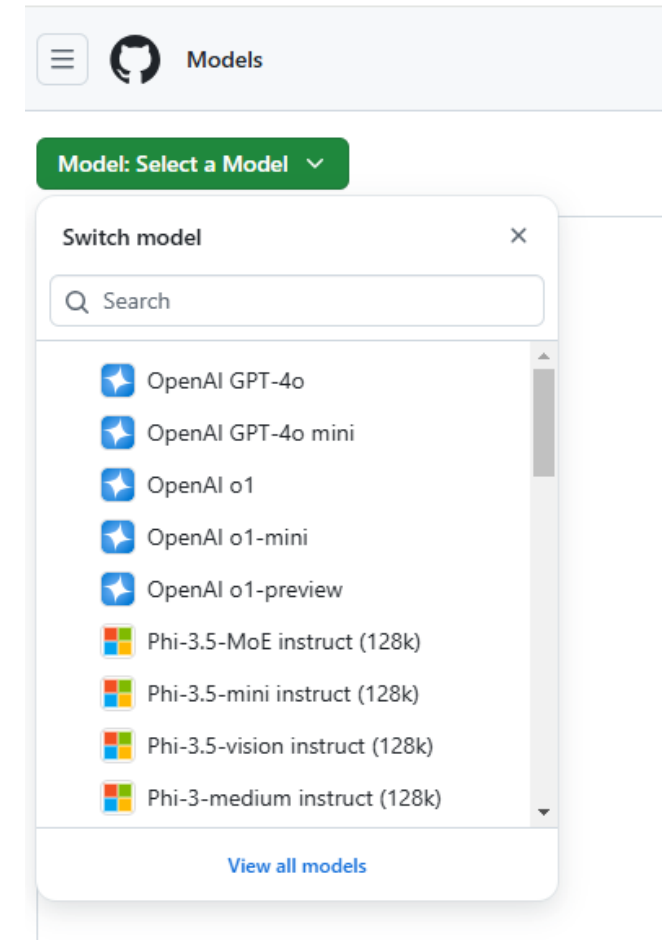
<https://openrouter.ai/models>

GitHub Models

- You can use GitHub Models to find and experiment with AI models for free.

1. Go to github.com/marketplace/models.

2. Click **Model: Select a Model** at the top left of the page.



- Visit <https://github.com/marketplace/models/azure-openai/gpt-4-1-mini/playground>

Q Type to search

Preview

Give feedback

Preset: Default

<> Get API key

Input: 0/0 • Output: 0/0 • 0ms

Parameters

Details

it and image tasks.

Can you explain the concept of time dilation in physics?

About

An affordable, efficient AI solution for diverse text and image tasks.

Context

131k input · 4k output

Training date

Oct 2023

Rate limit tier

[Low](#)

Provider support

[Azure support site](#)

Tags 4

Rate limits

The playground and free API usage are rate limited by requests per minute, requests per day, tokens per request, and concurrent requests. If you get rate limited, you will need to wait for the rate limit that you hit to reset before you can make more requests.

Low, high, and embedding models have different rate limits. To see which type of model you are using, refer to the model's information in GitHub Marketplace.

Rate limit tier	Rate limits	Free and Copilot Individual	Copilot Business	Copilot Enterprise
Low	Requests per minute	15	15	20
	Requests per day	150	300	450
	Tokens per request	8000 in, 4000 out	8000 in, 4000 out	8000 in, 8000 out
	Concurrent requests	5	5	8
High	Requests per minute	10	10	15
	Requests per day	50	100	150
	Tokens per request	8000 in, 4000 out	8000 in, 4000 out	16000 in, 8000 out
	Concurrent requests	2	2	4

- Follow the steps to get an API Key

Get API key

Language: Python
SDK: OpenAI SDK

Chapters

1. Create a personal access token
2. Install dependencies
3. Run a basic code sample
4. Explore more samples
5. Going beyond rate limits

Run with codepaces

Seriously, you'll be up and running in seconds. It will be great.

Run codepaces

Get started

1. Create a personal access token

To authenticate with the model you will need to generate a personal access token (PAT) in your GitHub settings or set up an Azure production key.

GitHub Free [Get developer key](#)

Access AI inference with your GitHub PAT. [Learn more about limits based on your plan.](#)

Azure AI by Azure Pay as you go [Get production key](#)

Access pay-as-you-go inference and more AI services on Azure.

You **do not need to give any permissions** to the token. Note that the token will be sent to a Microsoft service.

To use the code snippets below, create an environment variable to set your token as the key for the client code.

If you're using **bash**:

```
export GITHUB_TOKEN="<your-github-token-goes-here>"
```

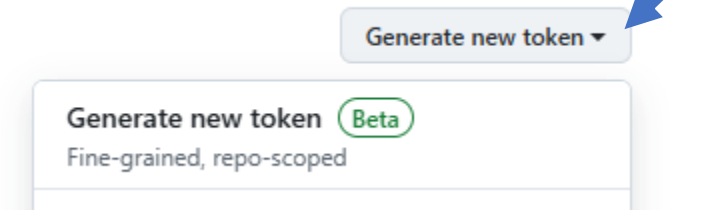
If you're in **powershell**:

Click “Generate Token”.
Copy the token and store securely.



No personal access token create

Need an API token for scripts or testing? Generate a personal acc
to the GitHub API.



- For security reasons, the API key should never be hardcoded in source code because it risks accidental exposure if the code is shared, published, or pushed to a public repository To secure the API key,
- We may store the API key in a .env file or environment variable
- Ensure the .env file is listed in .gitignore so it is not committed to version control.
- To load it from the .env file or environment variable in your code using a package like python-dotenv, then access it with `os.getenv("GITHUB_TOKEN")`.

New fine-grained personal access token Preview

Create a fine-grained, repository-scoped token suitable for personal API use and for using Git over HTTP.

Token name *

test_github_model

A unique name for this token. May be visible to resource owners or users with possession of the token.

Choose Python as language and OpenAI SDK.

Follow the remaining steps to configure the environment and run the sample program with the API key

Get started with Models in your codebase

Language: Python ▼

SDK: OpenAI SDK ▼

```
import os
from openai import OpenAI

token = os.environ["GITHUB_TOKEN"]
endpoint = "https://models.github.ai/inference"
model = "openai/gpt-4.1-mini"

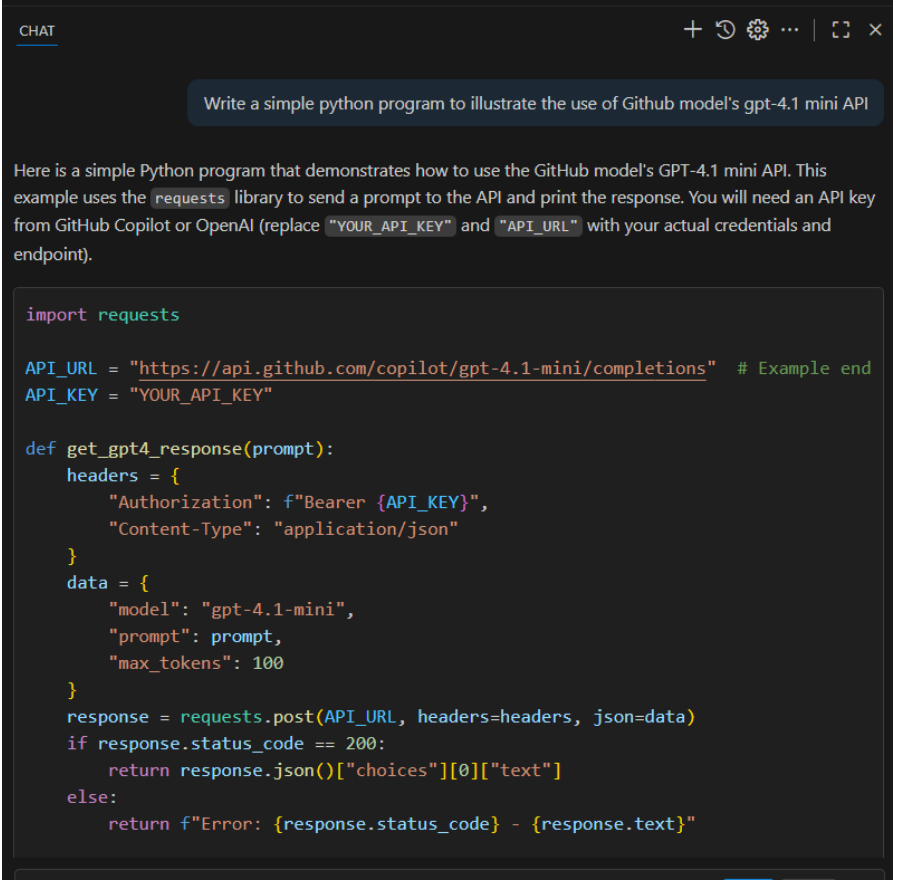
client = OpenAI(
    base_url=endpoint,
    api_key=token,
)

response = client.chat.completions.create(
    messages=[
        {
            "role": "system",
            "content": "You are a helpful assistant.",
        },
        {
            "role": "user",
            "content": "What is the capital of France?",
        }
    ],
    temperature=1.0,
    top_p=1.0,
    model=model
)

print(response.choices[0].message.content)
```

Limitations: Using LLM for coding

- Large language models (LLMs) are trained on large code corpora collected at a fixed point in time, LLMs often lack real-time updates on library evolution and API changes.
- As a result, they may suggest code that use deprecated libraries or outdated APIs that may no longer function correctly with the latest versions of libraries or frameworks.
- The generated code is more prone to hallucination, and might fail to compile or run, resulting in compatibility issues, bugs, or even security vulnerabilities.



```
CHAT
+ ⌚ ⚙️ ... | 🗨️ ✕

Write a simple python program to illustrate the use of Github model's gpt-4.1 mini API

Here is a simple Python program that demonstrates how to use the GitHub model's GPT-4.1 mini API. This example uses the requests library to send a prompt to the API and print the response. You will need an API key from GitHub Copilot or OpenAI (replace "YOUR_API_KEY" and "API_URL" with your actual credentials and endpoint).

import requests

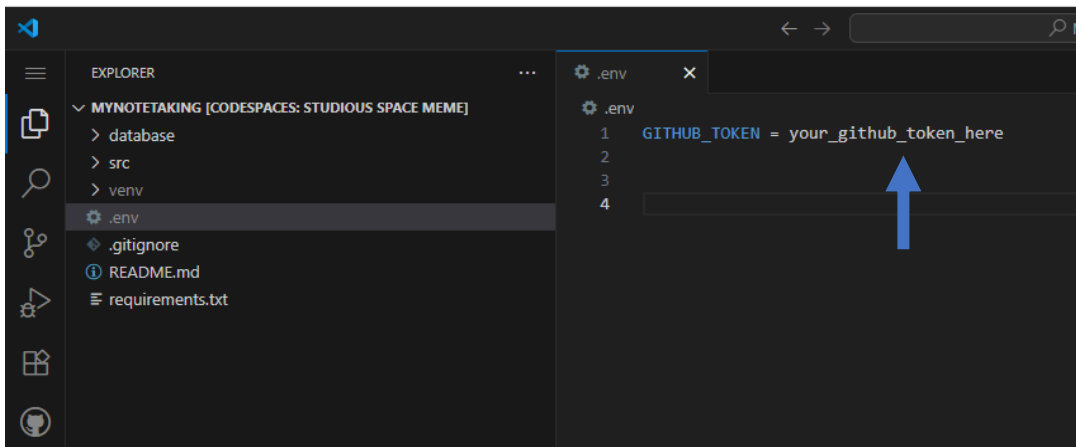
API_URL = "https://api.github.com/copilot/gpt-4.1-mini/completions" # Example end
API_KEY = "YOUR_API_KEY"

def get_gpt4_response(prompt):
    headers = {
        "Authorization": f"Bearer {API_KEY}",
        "Content-Type": "application/json"
    }
    data = {
        "model": "gpt-4.1-mini",
        "prompt": prompt,
        "max_tokens": 100
    }
    response = requests.post(API_URL, headers=headers, json=data)
    if response.status_code == 200:
        return response.json()["choices"][0]["text"]
    else:
        return f"Error: {response.status_code} - {response.text}"
```

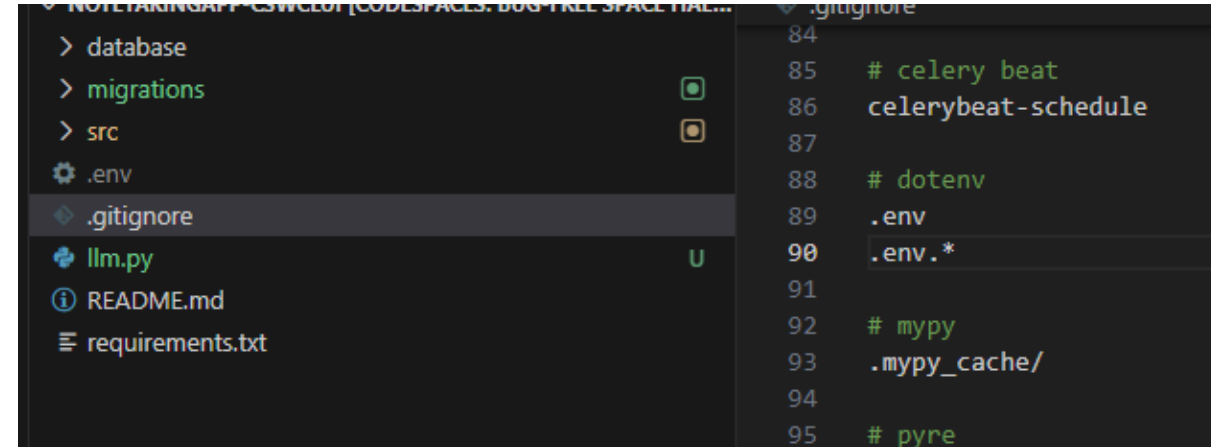
Is the code correct?

Storing API keys/credentials

- Store the GitHub Model Credential in .env file
- Make sure the **.env** is specified in **.gitignore** file and the **.env** is not tracked by git.



This screenshot shows the VS Code Explorer on the left with the project structure expanded. The `.env` file is highlighted in the Explorer. On the right, the `.env` file is open in the editor, showing the line `GITHUB_TOKEN = your_github_token_here`. A blue arrow points to this line.



This screenshot shows the VS Code Explorer on the left with the `.gitignore` file highlighted. On the right, the `.gitignore` file is open in the editor, showing the following content:

```
84  
85 # celery beat  
86 celerybeat-schedule  
87  
88 # dotenv  
89 .env  
90 .env.*  
91  
92 # mypy  
93 .mypy_cache/  
94  
95 # pyre
```

Define the following program

src/llm.py

Instead of asking LLM to complete the feature on its own, we write our code to guide LLM in the code generation.

```
# import libraries
[Add your code here]

load_dotenv() # Loads environment variables from .env
token = os.environ["GITHUB_TOKEN"]
endpoint = "https://models.github.ai/inference"
model = "openai/gpt-4.1-mini"

# A function to call an LLM model and return the response
def call_llm_model(model, messages, temperature=1.0, top_p=1.0):
    client = OpenAI(base_url=endpoint, api_key=token)
    response = client.chat.completions.create(
        messages=messages,
        temperature=temperature, top_p=top_p, model=model)
    return response.choices[0].message.content

# A function to translate to target language
[Add your code here]

# Run the main function if this script is executed
[Add your code here]
```

Run the program

- Add the library in requirements.txt
 - openai==1.106.1
 - dotenv==0.9.9
- Install the library in the virtual environment
 - pip install -r requirements.txt
- Run the app

```
• (venv) @cswclui →/workspaces/MyNoteTaking (ref) $ python src/llm.py  
你好，你怎么样？  
○ (venv) @cswclui →/workspaces/MyNoteTaking (ref) $ █
```

Implement the feature

- Add `src/llm.py` to Copilot chat and use it for context
- Use the copilots “Agent mode” to reference the code and use the defined function to implement the translation feature by modifying the frontend and backend API.

Commit change in Dev branch

- Use the GitHub Copilot Agent mode to add the translation function to the App
- Stage the changes
- Commit with message **"feat: Translate notes"**

```
# Create and checkout to a new branch named "Dev"  
# If you already have the branch created before, use "git checkout Dev" to change to the  
branch
```

```
git checkout -b Dev
```

```
# Stage your changes and commit
```

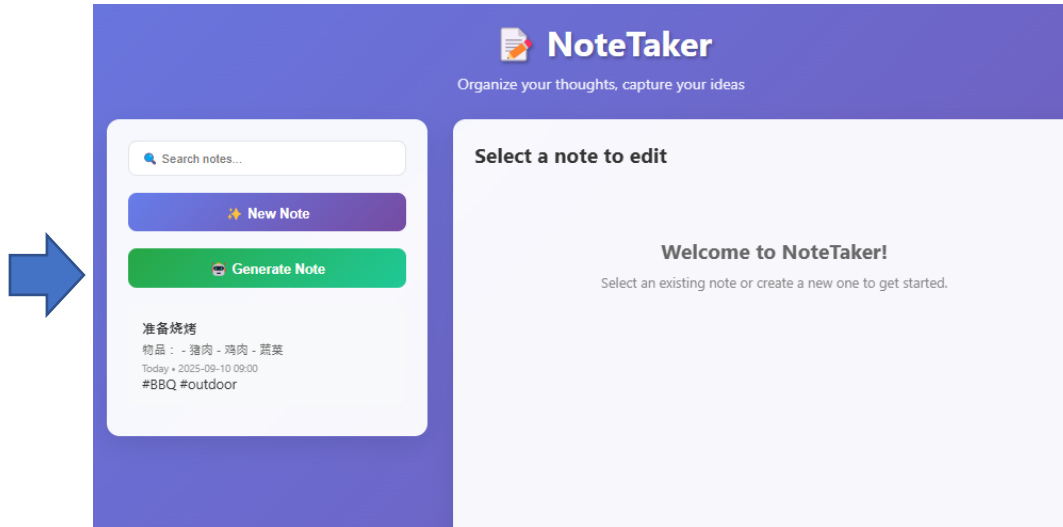
```
git add .
```

```
git commit -m "feat: Translate notes"
```

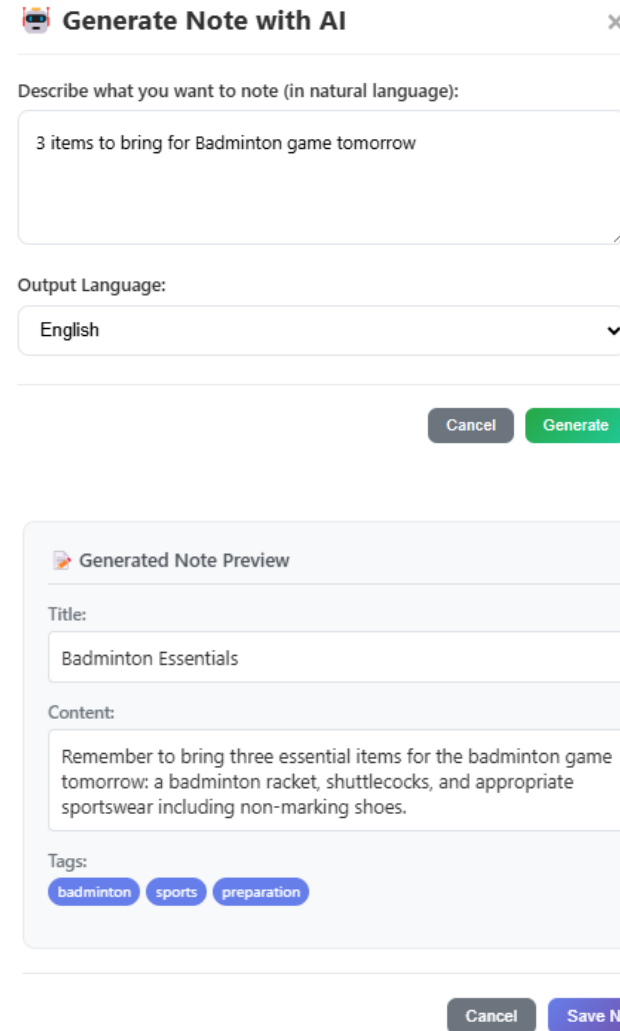
```
# Push the new branch and commit to GitHub
```

```
git push origin Dev
```

Add feature: “Generate Notes”



Allow the user to input the notes in natural language.
LLM is used to generate the title, contents of the notes
with three suggested tags





Organize your thoughts, capture your ideas

Search notes...

New Note

Generate Note

Badminton Essentials

Remember to bring three essential items for the badminton game tomorrow: a badminton...

Today

#badminton #sports #preparation

准备烧烤

物品： - 猪肉 - 鸡肉 - 蔬菜

Today • 2025-09-10 09:00

#BBQ #outdoor

Badminton Essentials

Chinese

Translate

Save

Delete

Title

Badminton Essentials

Content

Remember to bring three essential items for the badminton game tomorrow: a badminton racket, shuttlecocks, and appropriate sportswear including non-marking shoes.

Tags (comma-separated)

badminton, sports, preparation

Event Date

mm/dd/yyyy



Event Time

--:--:--



The AI generated notes with title, content, and tags

Add these fields to your app and DB first (if you have not done so in the last lab exercise.

- First, we implement the helper function to generate the notes in structured JSON format.

```
system_prompt = '''
Extract the user's notes into the following structured fields:
1. Title: A concise title of the notes less than 5 words
2. Notes: The notes based on user input written in full sentences.
3. Tags (A list): At most 3 Keywords or tags that categorize the content of the notes.

Output in JSON format without ```json. Output title and notes in the language: {lang}.
Example:
Input: "Badminton tmr 5pm @polyu".
Output:
{{
  "Title": "Badminton at PolyU",
  "Notes": "Remember to play badminton at 5pm tomorrow at PolyU.",
  "Tags": ["badminton", "sports"]
}}

...
'''
```

Alternatively, you may also store the system prompt in a file (to facilitating easy customization).


```

def process_user_notes(language, user_input):
    system_prompt_filled = system_prompt.format(
        lang=language
    )

    messages = [
        {
            "role": "system",
            "content": system_prompt_filled,
        },
        {
            "role": "user",
            "content": user_input,
        }
    ]

    response_content = call_llm_model(model, messages)
    return response_content

# Run the main function if this script is executed
if __name__ == "__main__":
    result = process_user_notes("Chinese", "Get up tomorrow 7am")
    print(result)

    result = process_user_notes("English", "Learn python programming and docker")
    print(result)

```

Run the program

```

}
• (venv) @cswclui →/workspaces/MyNoteTaking (ref) $ python src/llm.py
{
  "Title": "明天7点起床",
  "Notes": "记得明天早上7点起床。",
  "Tags": ["提醒", "起床时间"]
}
{
  "Title": "Learn Python and Docker",
  "Notes": "Plan to learn Python programming and understand how to use Docker.",
  "Tags": ["python", "docker", "programming"]
}

```

Use the agent mode (Claude 4) and ask AI to implement the feature by using the above helper function.

Exercise: Extend the “Generate notes” function to extract the date and time (if available)

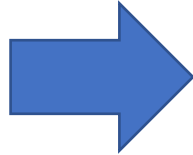
 **Generate Note with AI** ×

Describe what you want to note (in natural language):

lunch 1230pm tomorrow

Output Language:

English ▼



Lunch appointment Chinese Translate Save Delete

Title

Lunch appointment

Content

Remember to have lunch tomorrow at 12:30 pm.

Tags (comma-separated)

lunch, appointment

Event Date

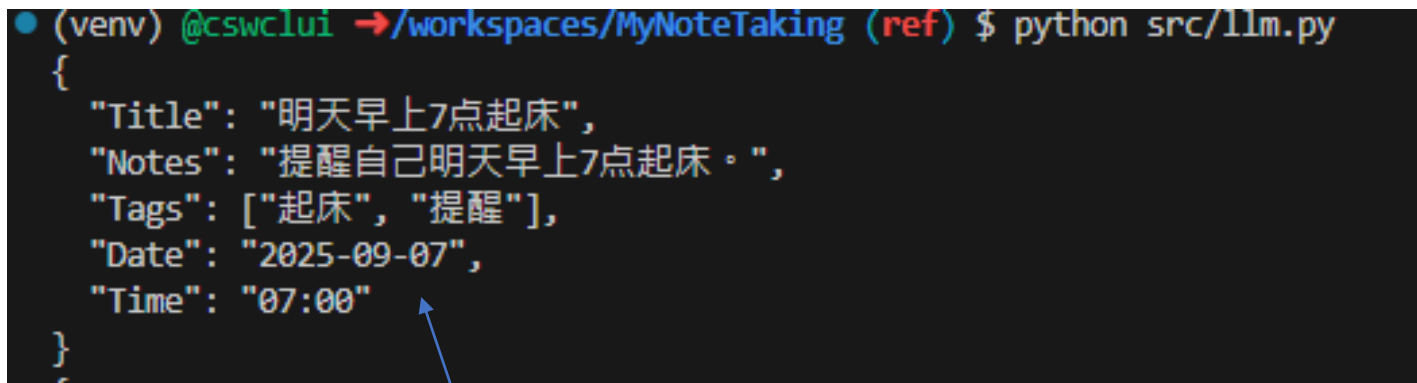
09/07/2025 

Event Time

12:30 PM 

You can first modify the helper function to extract the data and time from user input. Then, use it as the context for the Copilot Agent to modify the app.

```
result = process_user_notes("Chinese", "Get up tomorrow 7am")  
print(result)
```



```
• (venv) @cswclui →/workspaces/MyNoteTaking (ref) $ python src/llm.py  
{  
  "Title": "明天早上7点起床",  
  "Notes": "提醒自己明天早上7点起床。",  
  "Tags": ["起床", "提醒"],  
  "Date": "2025-09-07",  
  "Time": "07:00"  
}
```

LLM doesn't have any knowledge about the current date/time.
How can we let the LLM knows the date for tomorrow?

Copilot instruction

- Defined in a markdown file named **.github/copilot-instructions.md**
 - Natural language guidance to help Github Copilot generate code and responses better tailored to a project's specific needs, team workflows, coding standards, and preferences.
 - E.g. You may define your coding style preference, how you want APIs called, naming patterns you want followed, etc
- The Copilot Agent will first read the file to understand the project contexts/rules before completing the actual task
- Purpose
 - Help align AI-generated code with human expectations, project standards, and team workflows.
 - Help the AI understand project-specific context, business logic, and domain knowledge

Example 1

This is a Next.js-based travel application with TypeScript that helps users search for trips, manage bookings, view travel guides, and track points. The application uses React components, server components, and client components as part of the Next.js App Router architecture. Please follow these guidelines when contributing:

Code Standards

Required Before Each Commit

- Run ``npm run lint`` to ensure code follows project standards
- Make sure all components follow Next.js App Router patterns
- Client components should be marked with `'use client'` when they use browser APIs or React hooks
- When adding new functionality, make sure you update the README
- Make sure that the repository structure documentation is correct and accurate in the Copilot Instructions file
- Ensure all tests pass by running ``npm run test`` in the terminal

TypeScript and React Patterns

- Use TypeScript interfaces/types for all props and data structures
- Follow React best practices (hooks, functional components)
- Use proper state management techniques
- Components should be modular and follow single-responsibility principle

Styling

- You must prioritize using Tailwind CSS classes as much as possible. If needed, you may define custom Tailwind Classes / Styles. Creating custom CSS should be the last approach.

Development Flow

- Install dependencies: ``npm install``
- Development server: ``npm run dev``
- Build: ``npm run build``
- Test: ``npm run test``
- Lint: ``npm run lint``

Development Flow

- Install dependencies: ``npm install``
- Development server: ``npm run dev``
- Build: ``npm run build``
- Test: ``npm run test``
- Lint: ``npm run lint``

Repository Structure

- ``app/``: Next.js App Router pages and layouts organized by route
- ``components/``: Reusable React components
 - ``components/ui/``: UI components (buttons, inputs, etc.)
 - ``components/__tests__/``: Component tests
- ``lib/``: Core logic and services
 - ``lib/data/``: Data models and mock data
 - ``lib/types/``: TypeScript type definitions
- ``public/``: Static assets
- ``tests/``: Test files and test utilities
- ``README.md``: Project documentation

Key Guidelines

1. Make sure to evaluate the components you're creating, and whether they need `'use client'`
2. Images should contain meaningful alt text unless they are purely for decoration. If they are for decoration only, a null (empty) alt text should be provided (`alt=""`) so that the images are ignored by the screen reader.
3. Follow Next.js best practices for data fetching, routing, and rendering
4. Use proper error handling and loading states
5. Optimize components and pages for performance

Example 2

← Project Awareness & Context

- Always read `PLANNING.md` at the start of a new conversation to understand the project's architecture, goals, style, and constraints.
- Check `TASK.md` before starting a new task. If the task isn't listed, add it with a brief description and today's date.
- Use consistent naming conventions, file structure, and architecture patterns as described in `PLANNING.md`.
- Use `venv_linux` (the virtual environment) whenever executing Python commands, including for unit tests.

🗂 Code Structure & Modularity

- Never create a file longer than 500 lines of code. If a file approaches this limit, refactor by splitting it into modules or helper files.
- Organize code into clearly separated modules, grouped by feature or responsibility. For agents this looks like:
 - `agent.py` - Main agent definition and execution logic
 - `tools.py` - Tool functions used by the agent
 - `prompts.py` - System prompts
- Use clear, consistent imports (prefer relative imports within packages).
- Use clear, consistent imports (prefer relative imports within packages).
- Use `python_dotenv` and `load_env()` for environment variables.

🧪 Testing & Reliability

- Always create Pytest unit tests for new features (functions, classes, routes, etc).
- After updating any logic, check whether existing unit tests need to be updated. If so, do it.
- Tests should live in a `/tests` folder mirroring the main app structure.
 - Include at least:
 - 1 test for expected use
 - 1 edge case
 - 1 failure case

✅ Task Completion

- Mark completed tasks in `TASK.md` immediately after finishing them.
- Add new sub-tasks or TODOs discovered during development to `TASK.md` under a "Discovered During Work" section.

👤 Style & Conventions

- Use Python as the primary language.
- Follow PEP8, use type hints, and format with `black`.
- Use `pydantic` for data validation.
- Use `FastAPI` for APIs and `SQLAlchemy` or `SQLModel` for ORM if applicable.
- Write docstrings for every function using the Google style:

```
def example():  
    """  
    Brief summary.  
  
    Args:  
        param1 (type): Description.  
  
    Returns:  
        type: Description.  
    """
```

📖 Documentation & Explainability

- Update `README.md` when new features are added, dependencies change, or setup steps are modified.
- Comment non-obvious code and ensure everything is understandable to a mid-level developer.
- When writing complex logic, add an inline `# Reason:` comment explaining the why, not just the what.

🧠 AI Behavior Rules

- Never assume missing context. Ask questions if uncertain.
- Never hallucinate libraries or functions – only use known, verified Python packages.
- Always confirm file paths and module names exist before referencing them in code or tests.
- Never delete or overwrite existing code unless explicitly instructed to or if part of a task from `TASK.md`.